Assignment 14.4 Ai Assisted Coding

Htno:2303a51988

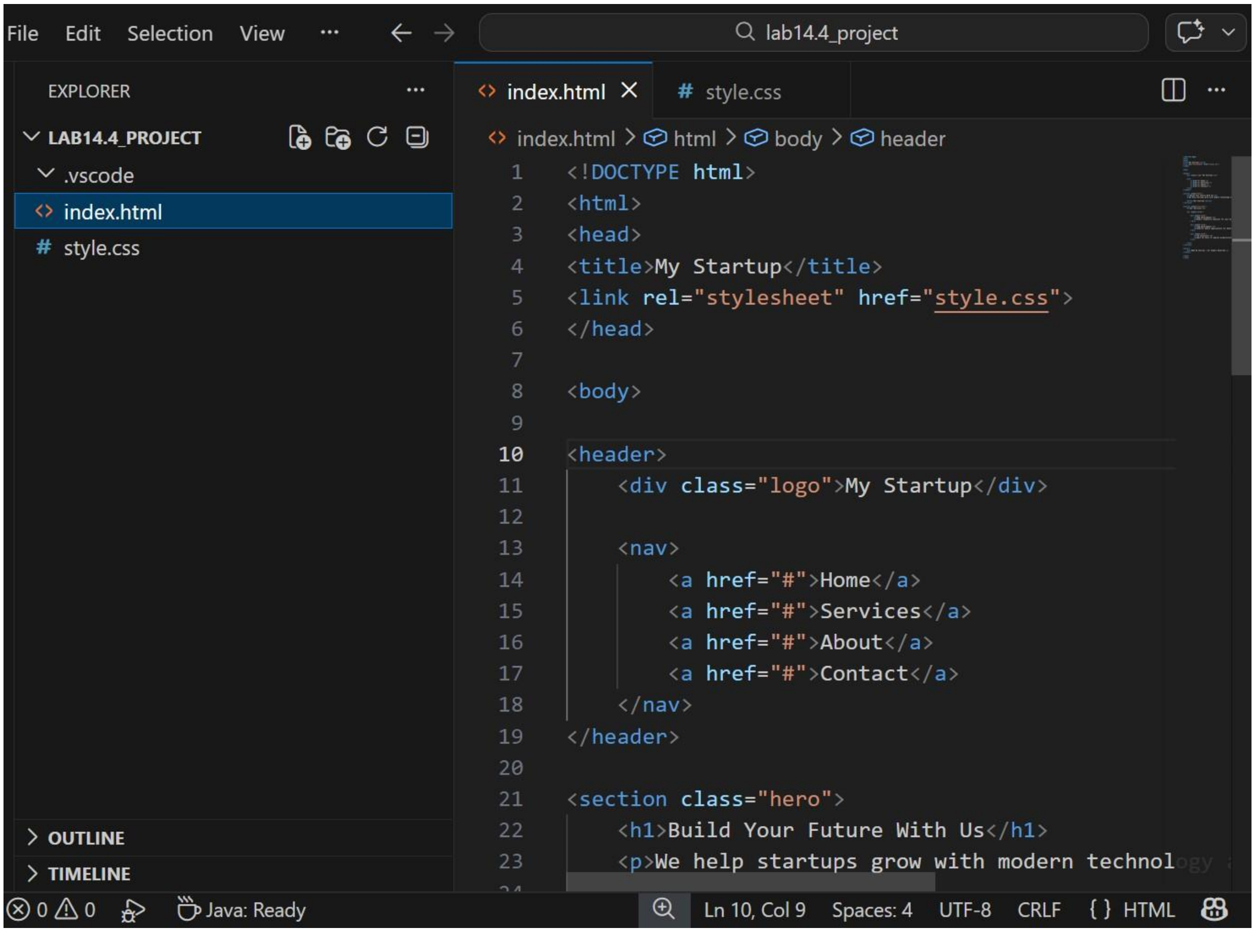
Btno:06

Task 1: Task 1: Responsive Homepage using HTML and CSS

Prompt: Generate a responsive homepage using HTML and CSS with a header, navigation menu, hero section, services section, and footer.

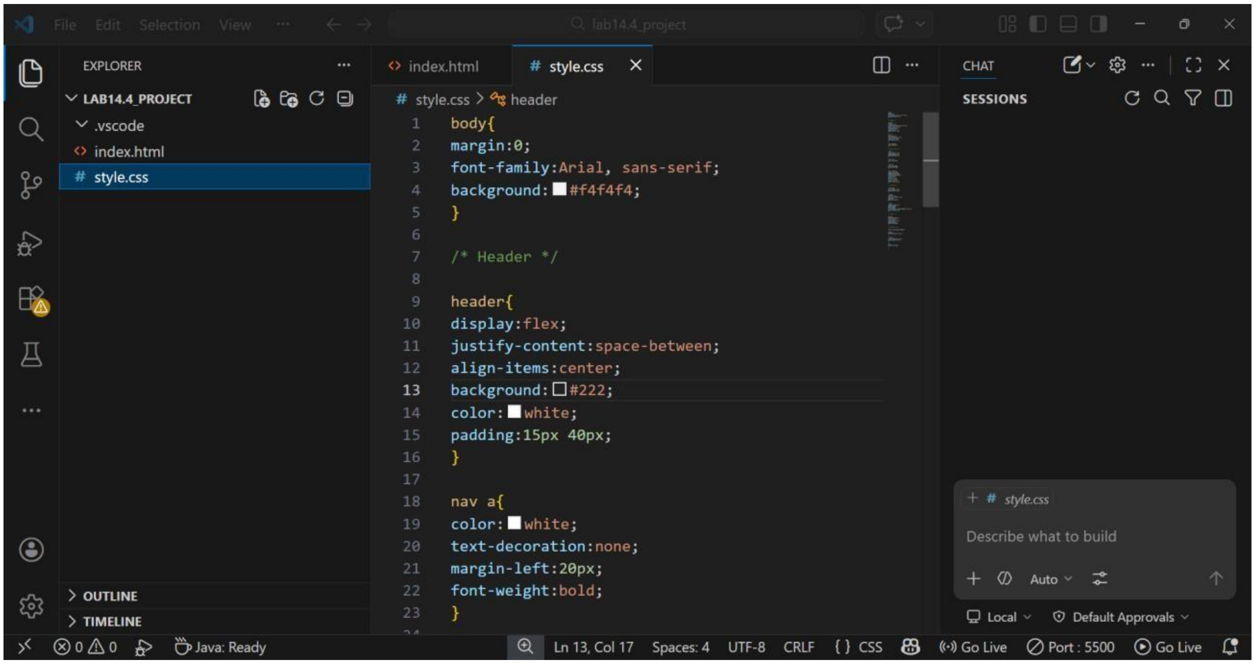
Use CSS media queries to make the layout responsive.

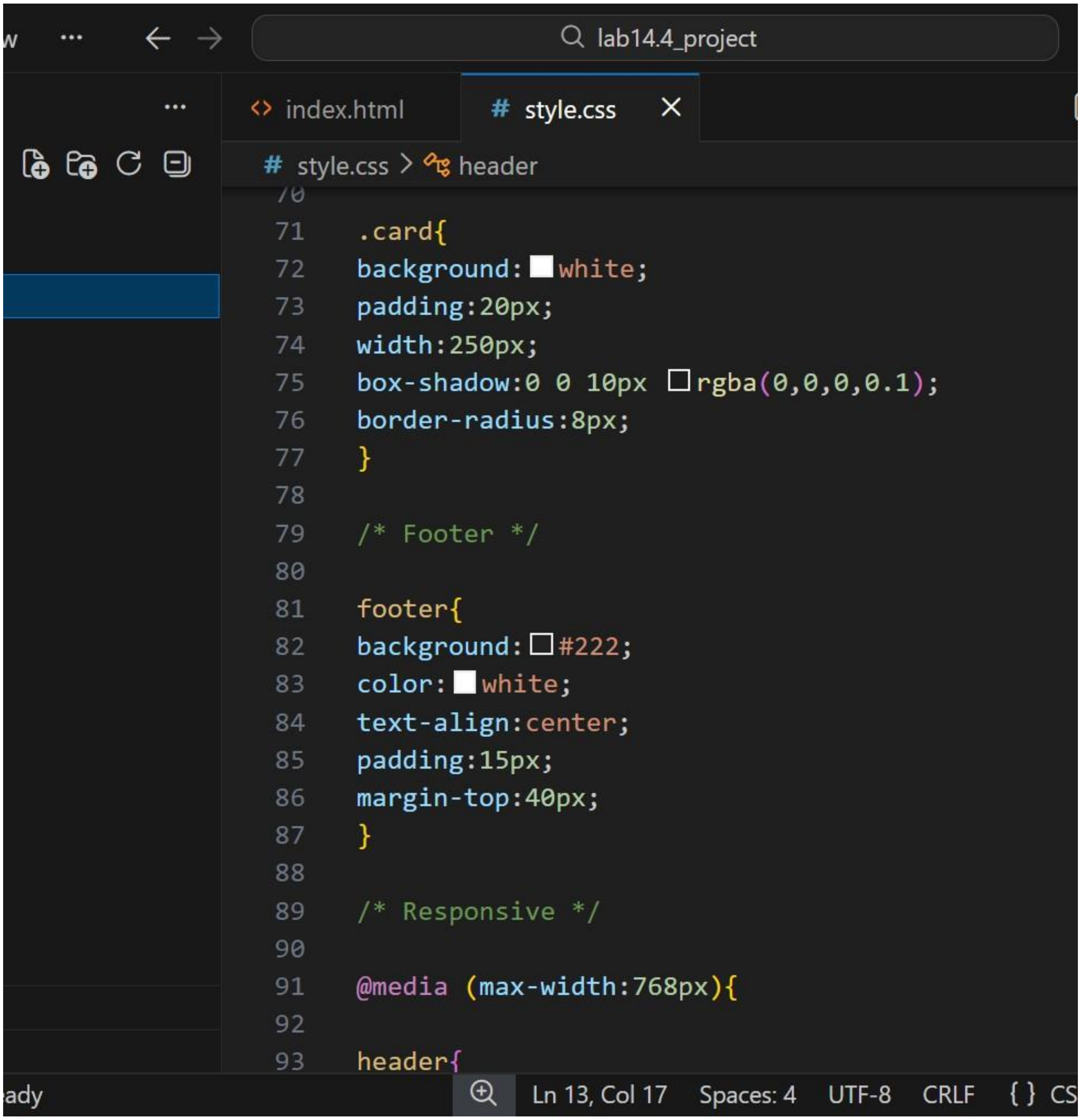
Codes: index ,style css


File Edit Selection View ···
lab14.4_project
EXPLORER
LAB14.4_PROJECT
.vscode
index.html
style.css
OUTLINE
TIMELINE
index.html
style.css
index.html > html > body > header
<!DOCTYPE html>
<html>
<head>
<title>My Startup</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header>
<div class="logo">My Startup</div>
<nav>
Home
Services
About
Contact
</nav>
</header>
<section class="hero">
<h1>Build Your Future With Us</h1>
<p>We help startups grow with modern technol
0 0 Java: Ready
Ln 10, Col 9 Spaces: 4 UTF-8 CRLF { } HTML

```html
<html>
<body>
<section class="services">
    <div class="cards">
        <div class="card">
            <h3>App Development</h3>
            <p>Powerful mobile applications for Andr
        </div>

        <div class="card">
            <h3>AI Solutions</h3>
            <p>Smart AI tools to improve productivity
        </div>

    </div>
</section>

<footer>
    <p>© 2026 My Startup | All Rights Reserved</p>
</footer>

</body>
</html>
```

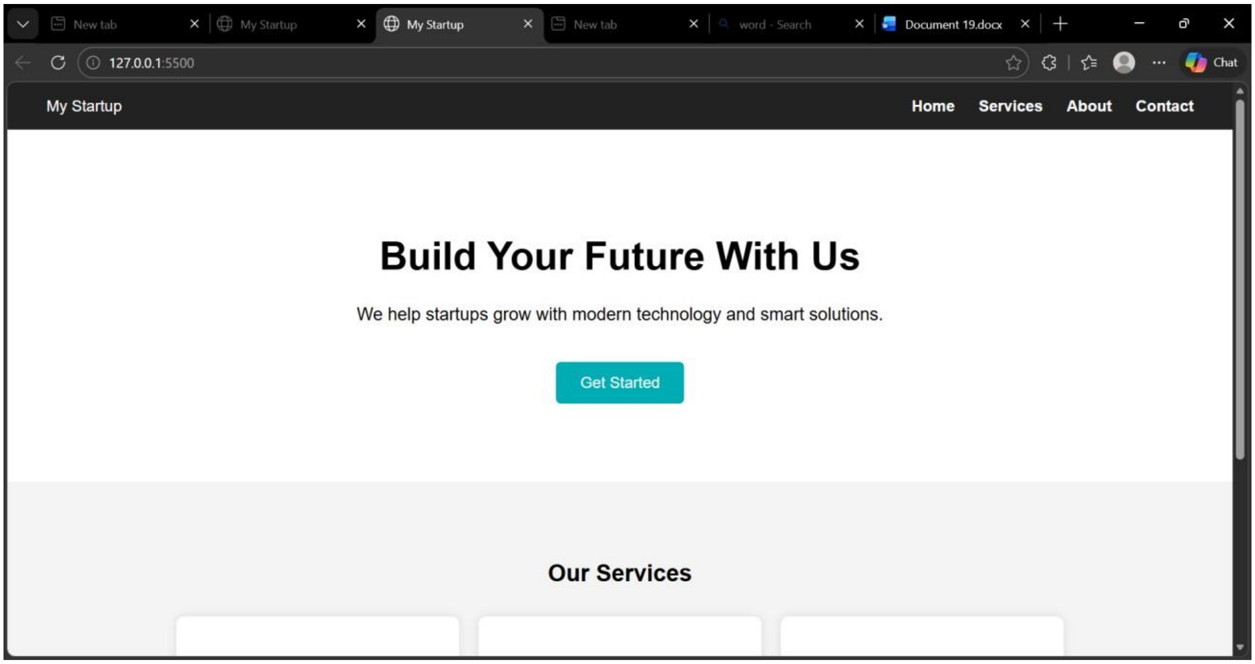
Style css:

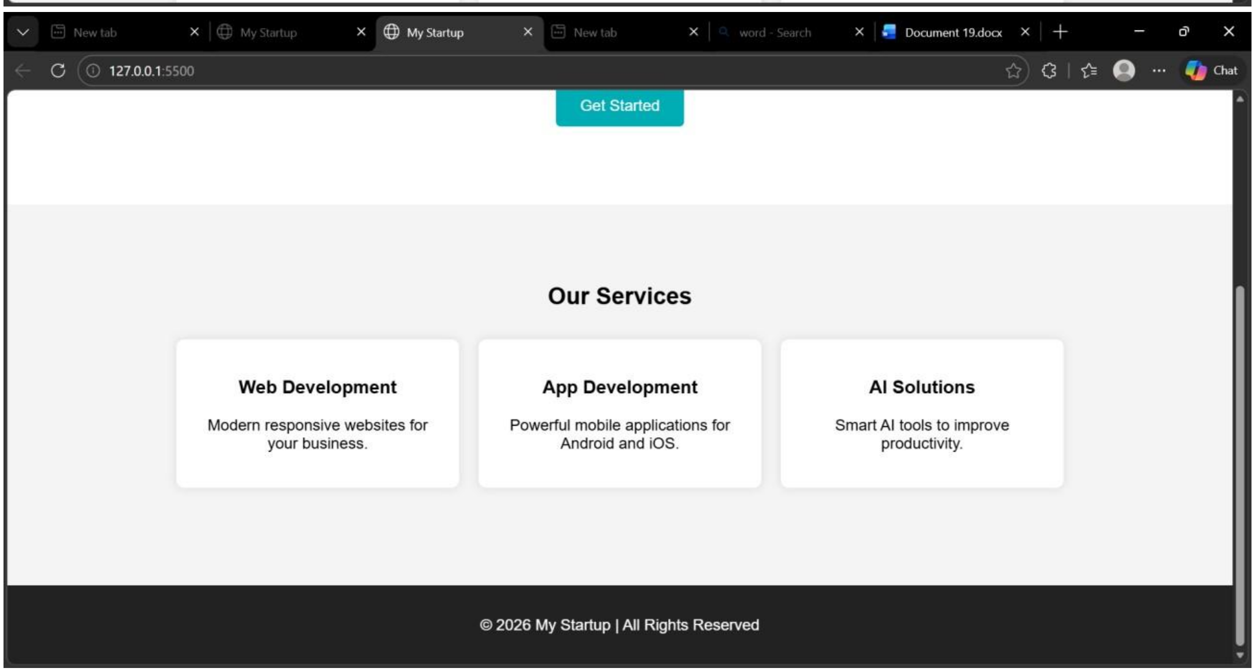

File Edit Selection View
lab14.4_project
EXPLORER
LAB14.4_PROJECT
.vscode
index.html
style.css
OUTLINE
TIMELINE
index.html
style.css
style.css > header
body{
margin:0;
font-family:Arial, sans-serif;
background:#f4f4f4;
}
/* Header */
header{
display:flex;
justify-content:space-between;
align-items:center;
background:#222;
color:white;
padding:15px 40px;
}
nav a{
color:white;
text-decoration:none;
margin-left:20px;
font-weight:bold;
}
CHAT
SESSIONS
style.css
Describe what to build
Auto
Local
Default Approvals
Java: Ready
Ln 13, Col 17
Spaces: 4
UTF-8
CRLF
{ } CSS
Go Live
Port : 5500
Go Live



lab14.4_project
index.html
style.css
style.css > header
.card{
background: white;
padding:20px;
width:250px;
box-shadow:0 0 10px rgba(0,0,0,0.1);
border-radius:8px;
}
/* Footer */
footer{
background: #222;
color: white;
text-align:center;
padding:15px;
margin-top:40px;
}
/* Responsive */
@media (max-width:768px){
header{
Ln 13, Col 17
Spaces: 4
UTF-8
CRLF

Output:





Explanation:

-->This task creates a responsive homepage using HTML and CSS.

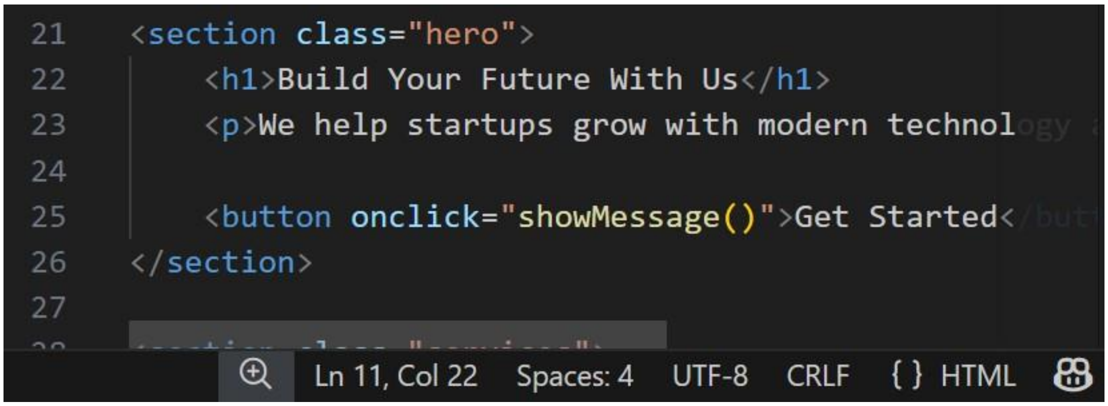
-->The webpage contains a header with navigation links, a hero section, -->service cards, and a footer. CSS is used for styling and media queries are used to make the webpage responsive for different screen sizes.

Task 2: Button Interaction using JavaScript

Prompt: Add a call-to-action button to the webpage.

When the user clicks the button, display an alert message using JavaScript without reloading the page.

Codes: index.html – edited part

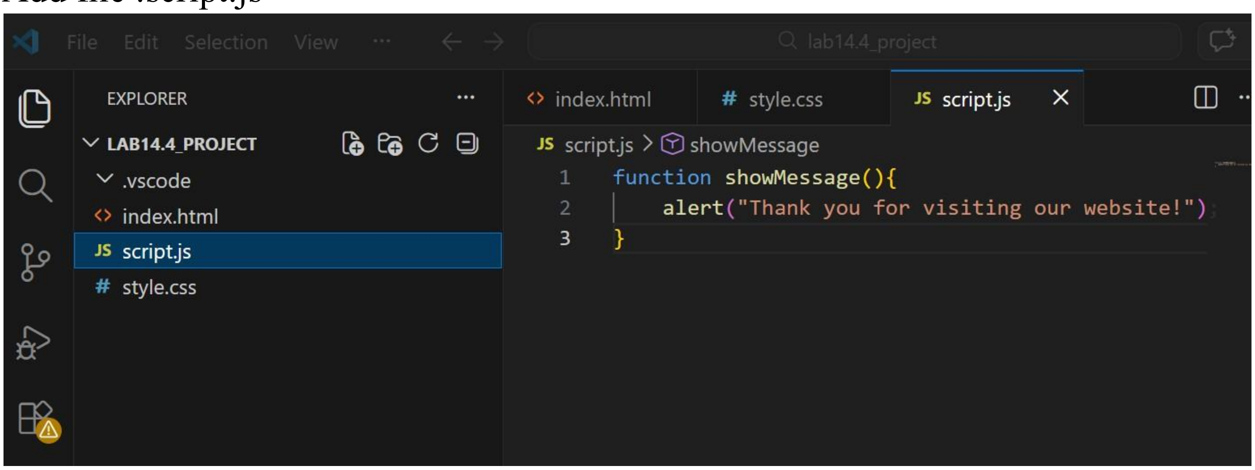


```html
<section class="hero">
    <h1>Build Your Future With Us</h1>
    <p>We help startups grow with modern technol

    <button onclick="showMessage()">Get Started<
</section>
```


Java Script connection:

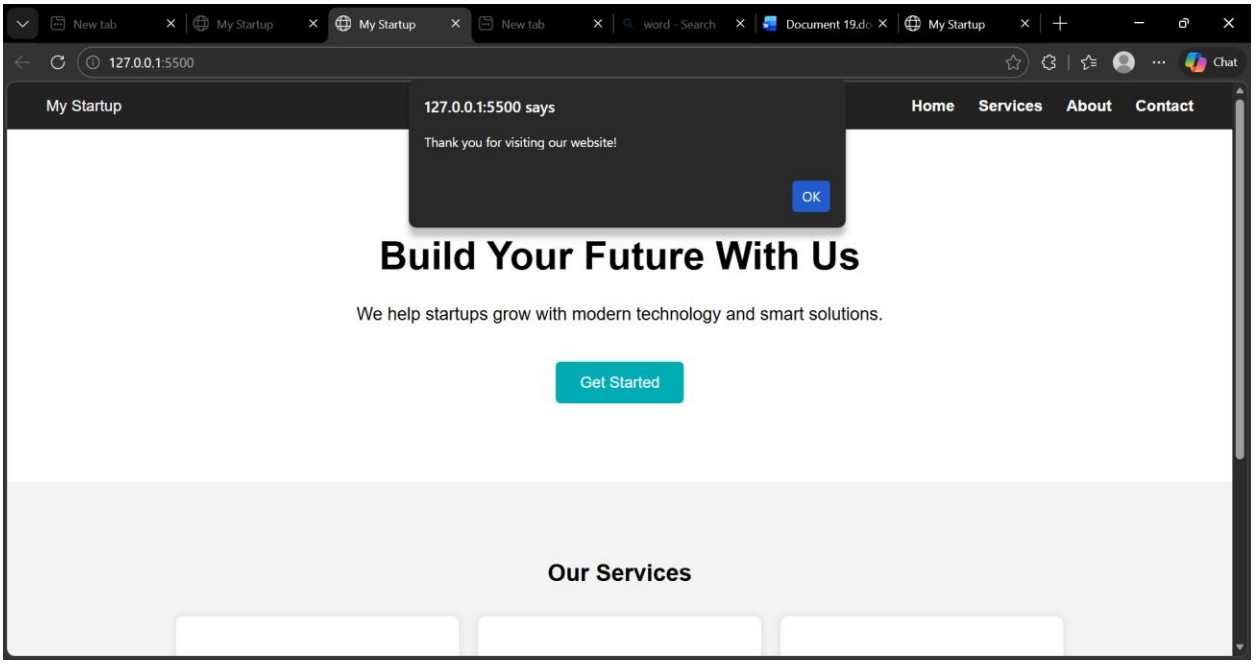
```html
<script src="script.js"></script>
```

Add file :script.js




```javascript
function showMessage(){
    alert("Thank you for visiting our website!")
}
```

Output:



Explanation:

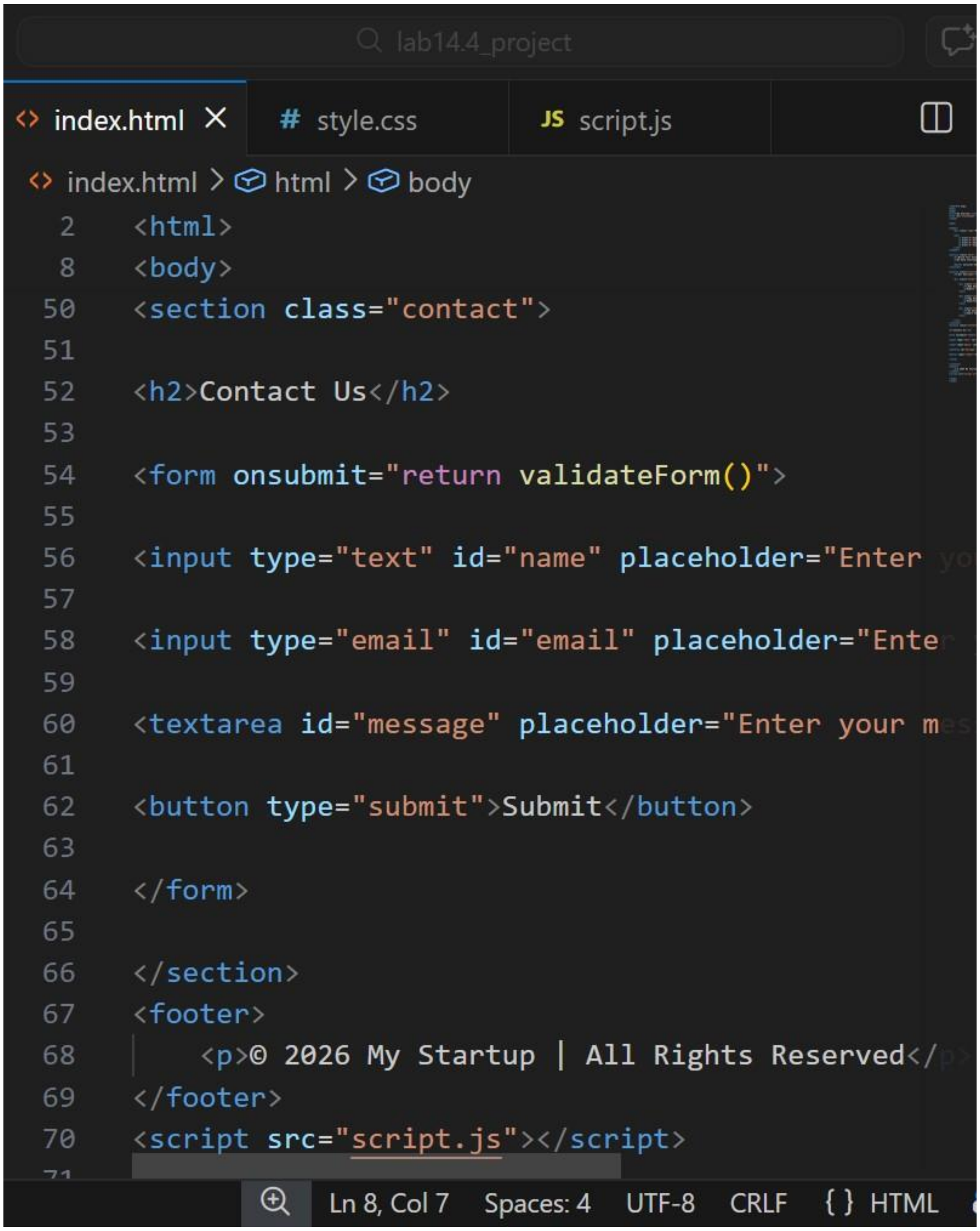
The button interaction is implemented using JavaScript.
When the user clicks the Get Started button, the showMessage() function is triggered and an alert message is displayed on the screen.

Task 3: Contact Form Validation using JavaScript

Prompt: Create a contact form using HTML with fields for Name, Email, and Message.

Use JavaScript to validate the form by checking empty fields and verifying the email format before submission.

Codes: index.html – Contact Form Section

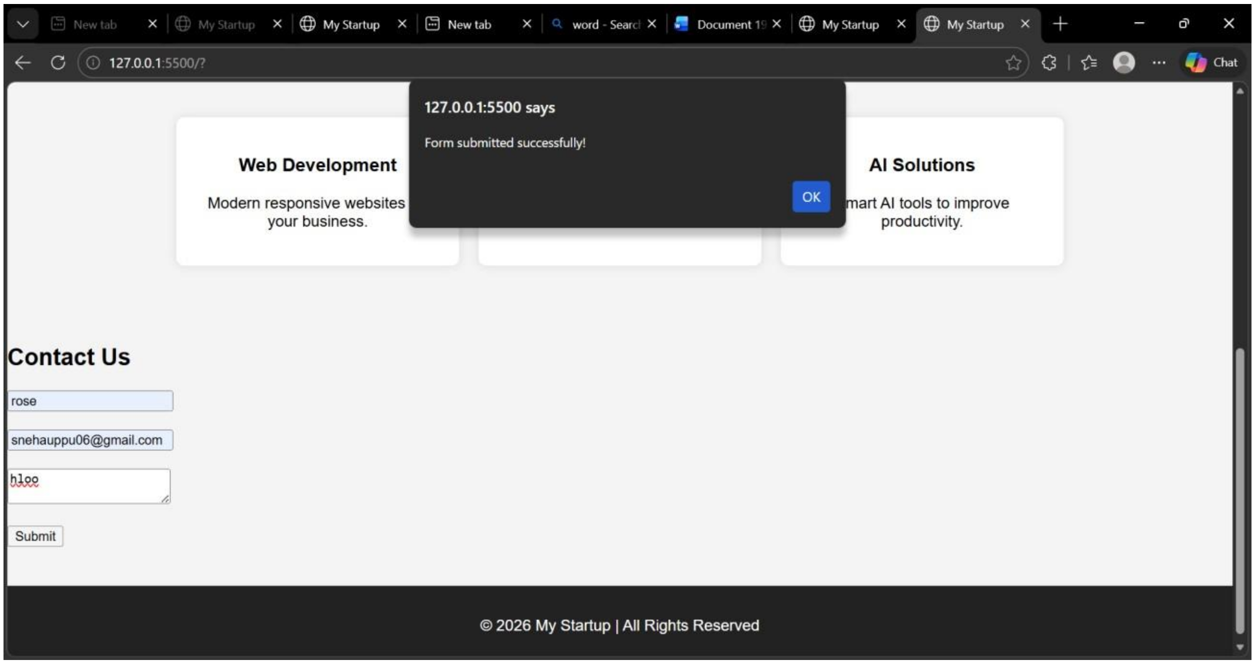


```html
<html>
<body>
<section class="contact">

<h2>Contact Us</h2>

<form onsubmit="return validateForm()">

<input type="text" id="name" placeholder="Enter yo

<input type="email" id="email" placeholder="Enter 

<textarea id="message" placeholder="Enter your mes

<button type="submit">Submit</button>

</form>

</section>
<footer>
    <p>© 2026 My Startup | All Rights Reserved</p>
</footer>
<script src="script.js"></script>
```

2)script.js – add below previous code:

```javascript
function validateForm(){

let name = document.getElementById("name").value;
let email = document.getElementById("email").value;
let message = document.getElementById("message").valu

if(name=="" || email=="" || message==""){
alert("Please fill all fields");
return false;
}

alert("Form submitted successfully!");
return true;

}
```

Output:



Expalantion:

This task creates a **contact form with Name, Email, and Message fields**.
JavaScript validation checks whether the fields are empty and displays an alert message before submitting the form.

Task 4: Dynamic Product List using JavaScript

Prompt: Create a dynamic product list where users can **add new items and remove items** from the list using JavaScript buttons.

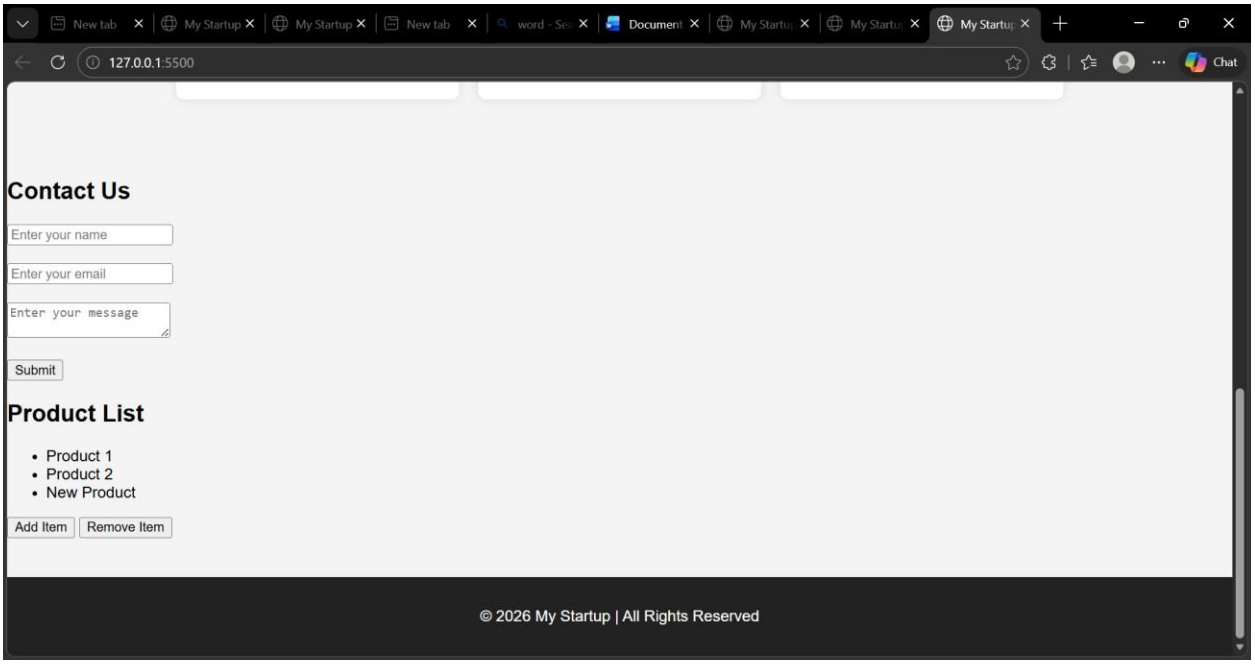
Code:

```html
<html>
<body>
<section class="list">

<h2>Product List</h2>

<ul id="itemList">
<li>Product 1</li>
<li>Product 2</li>
</ul>

<button onclick="addItem()">Add Item</button>
<button onclick="removeItem()">Remove Item</button>

</section>
<footer>
    <p>© 2026 My Startup | All Rights Reserved</p>
</footer>
<script src="script.js"></script>

</body>
</html>
```

2)Java Script Add Item:

```javascript
}
function addItem(){

let list = document.getElementById("itemList");
let newItem = document.createElement("li");

newItem.textContent = "New Product";

list.appendChild(newItem);

}

function removeItem(){

let list = document.getElementById("itemList");

if(list.lastElementChild){
list.removeChild(list.lastElementChild);
}

}
```

Output:



Explanation:

This program creates a dynamic product list where users can add or remove items using buttons. JavaScript DOM functions like `createElement()` and `appendChild()` are used to modify the list.

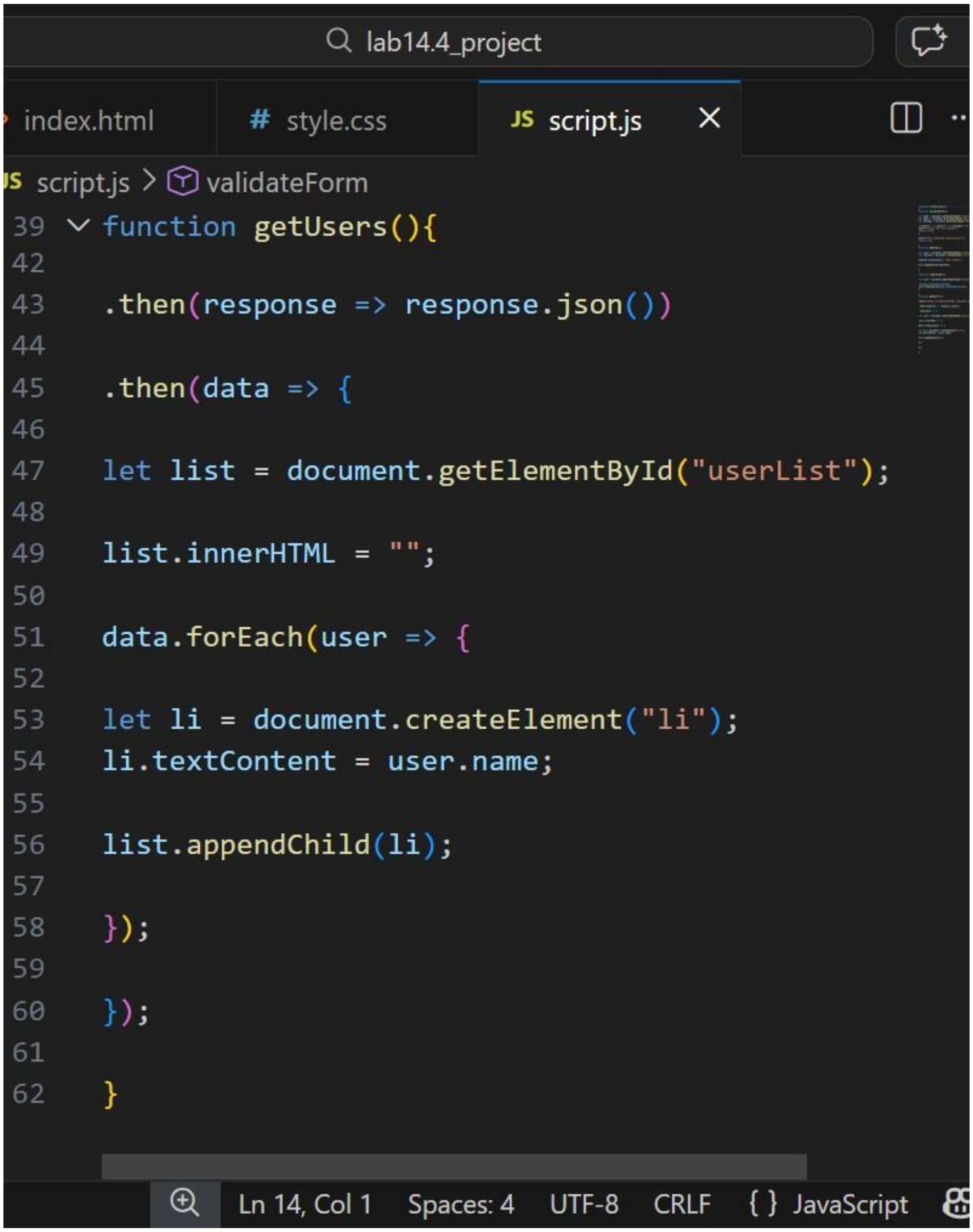
Task 5: Fetch Data using JavaScript API

Prompt: Create a webpage that **fetches data from an API using JavaScript Fetch API and displays it on the webpage dynamically**.

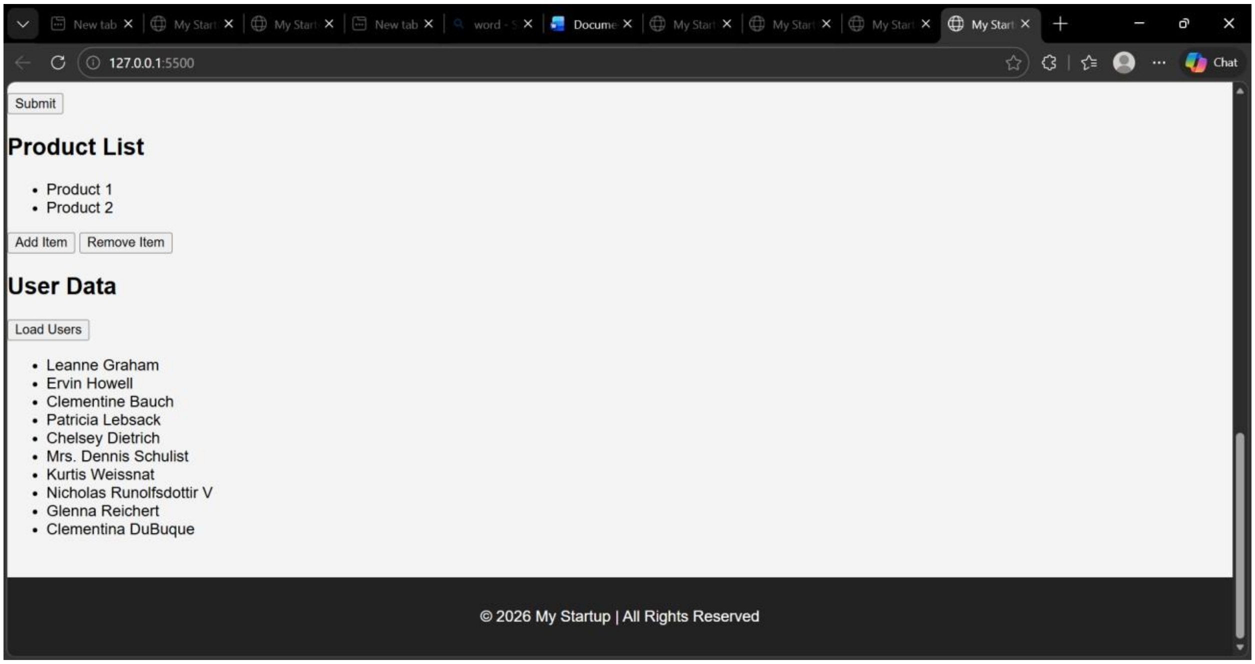
Codes: Index .Html

```html

</section>
<section class="api">

<h2>User Data</h2>

<button onclick="getUsers()">Load Users</button>

<ul id="userList"></ul>

</section>
<footer>
    <p>© 2026 My Startup | All Rights Reserved</p>
</footer>
<script src="script.js"></script>
```

2)Java Script(Load Users):


lab14.4_project
index.html
style.css
JS script.js
JS script.js > validateForm
function getUsers(){
.then(response => response.json())
.then(data => {
let list = document.getElementById("userList");
list.innerHTML = "";
data.forEach(user => {
let li = document.createElement("li");
li.textContent = user.name;
list.appendChild(li);
});
});
}
Ln 14, Col 1
Spaces: 4
UTF-8
CRLF
{ } JavaScript

Output:



Explanation:

The Fetch API is used to retrieve user data from an external API. JavaScript dynamically creates list elements and displays the user names on the webpage. The JavaScript code processes this data and dynamically creates list elements to display the user names on the webpage. This allows the webpage to update content without reloading the page.